

Vibe Project Generator 使用指南

本文档介绍如何与大模型交互，以触发 **Vibe Project Generator Skill** 自动生成项目。

一、什么是 Vibe Project Generator

Vibe Project Generator 是一个 AI Skill，它能够根据用户的自然语言描述，自动生成完整的、可运行的项目结构。

技能位置： `.trae/skills/vibe-project-generator/SKILL.md`

核心能力：

- 理解项目需求
- 设计项目结构
- 生成完整代码
- 创建配置文件
- 编写启动脚本
- 生成使用文档

二、触发 Skill 的方式

方式一：描述项目需求（推荐）

直接用自然语言描述你想要的项目，大模型会自动识别并调用 Skill。

示例：

"我想开发一个GraphRAG的教学案例，使用Neo4j作为后端存储"

"创建一个基于Ollama的本地RAG系统"

"开发一个能够对比RAG和GraphRAG的演示系统，包括前端界面"

方式二：提供代码片段

给大模型一段代码，让它生成完整项目。

示例：

"根据以下代码片段，生成一个完整的可运行程序：

[粘贴你的代码片段]

"

方式三：指定技术栈

明确说明要使用的技术，大模型会据此生成合适的项目结构。

示例：

"用Flask + Neo4j + Ollama创建一个知识图谱问答系统"

"开发一个React前端的Python后端项目"

方式四：描述使用场景

说明你的具体使用场景，让大模型推荐合适的方案。

示例：

"我需要创建一个教学演示用的知识图谱系统"

"为公司内部创建一个技术调研的原型"

三、交互示例

示例 1：创建简单的 CLI 工具

你说：

帮我创建一个Python命令行工具，可以统计文件夹中的代码行数

大模型响应：

(检测到需求, 自动调用 vibe-project-generator Skill)

好的, 我来为你创建一个代码行统计工具项目。

这将包括:

- 主程序: line_counter.py
- 依赖配置: requirements.txt
- 启动脚本: start.sh
- 使用文档: README.md

正在生成项目结构...

生成的项目结构:

```
line-counter/
├── line_counter.py      # 主程序
├── requirements.txt     # 依赖配置
├── start.sh            # 启动脚本
└── README.md           # 使用文档
```

示例 2: 创建 Web 应用

你说:

我想开发一个待办事项Web应用, 使用Flask作为后端, React作为前端

大模型响应:

好的, 我来为你创建一个完整的待办事项Web应用。

项目结构:

- 后端: Flask REST API
- 前端: React 单页应用
- 数据库: SQLite (开发环境)

正在生成项目...

生成的项目结构:

```
todo-app/
├── backend/
│   ├── app.py          # Flask 应用
│   ├── models.py       # 数据模型
│   ├── requirements.txt # Python 依赖
│   └── config.py        # 配置文件
```

```
├── frontend/
│   ├── public/
│   │   ├── index.html
│   │   ├── style.css
│   │   └── app.js
│   ├── package.json
│   └── server.js          # 代理服务器
├── start.sh              # 启动脚本
├── stop.sh               # 停止脚本
└── README.md             # 使用文档
```

示例 3：创建教学案例

你说：

我想开发一个GraphRAG的教学案例，展示如何从文本中提取三元组并存入Neo4j

大模型响应：

完美，我来为你创建一个完整的GraphRAG教学案例。
这将包括三个层次的实现：

- 1. 内存版 GraphRAG (local_graphrag_ollama.py)
 - 使用 NetworkX 内存存储
 - 展示核心流程
- 2. Neo4j 版 GraphRAG (local_graphrag_neo4j.py)
 - 使用 Neo4j 持久化存储
 - 展示图数据库优势
- 3. 前后端对比系统
 - GraphRAG vs 普通 RAG 对比
 - Web 界面展示

正在生成项目...

生成的项目结构：

```
graphrag/
├── local_graphrag_ollama.py    # 内存版
├── local_graphrag_neo4j.py    # Neo4j版
├── backend/
│   ├── app.py                 # Flask API
│   └── requirements.txt
├── frontend/
│   └── public/
```

```
├── index.html
├── style.css
├── app.js
├── server.js
├── package.json
├── data/
│   ├── sample_documents.txt
│   └── test_questions.txt
├── start_full.sh
├── stop_full.sh
├── requirements.txt
└── README.md
```

四、关键触发词

使用以下词汇可以更有效地触发 Skill：

触发词	示例	说明
"我想开发"	"我想开发一个RAG系统"	明确的项目需求
"创建一个"	"创建一个对比演示系统"	指向项目创建
"帮我生成"	"帮我生成一个Flask项目"	请求生成
"能不能做个"	"能不能做个教学案例"	口语化需求
"这是代码片段"	"这是代码，请生成完整程序"	提供参考
"我需要"	"我需要一个Web服务"	说明需求
"帮我做"	"帮我做一个原型"	请求帮助创建

五、加速技巧

1. 指定输出位置

```
"在 /projects/rag-demo 目录下创建一个GraphRAG项目"
```

2. 说明运行环境

```
"使用Ollama和Neo4j，运行环境是macOS"
```

3. 明确功能范围

"只需要核心功能，不需要用户认证"

4. 提供示例数据

"使用这些文档作为示例：
[粘贴文档内容]"

5. 说明目标用户

"面向初学者的教学案例，不需要太复杂"

6. 指定输出格式

"生成一个可以作为PPT演示的项目"

六、完整的交互流程



大模型: "项目已生成完成!
运行方式: `./start_graphrag.sh`
访问地址: `http://localhost:3000`
如需调整, 请告诉我。"

七、Skill 的能力边界

✅ 可以生成

- 完整的项目结构
- 可运行的代码 (含错误处理)
- 依赖配置文件 (requirements.txt / package.json)
- 启动/停止脚本
- README 使用文档
- 基础的测试用例
- 配置文件 (config.py 等)

❌ 无法保证

- 100% 代码正确性 (需人工验证)
- 复杂架构的自动设计 (需要更多指导)
- 特定业务逻辑的完整性
- 第三方服务的可用性

💡 当不确定时

大模型会主动询问:

- 技术栈偏好
- 具体功能需求
- 项目规模预期
- 部署环境要求

八、最佳实践

1. 从简单开始

❌ "创建一个像Facebook一样的社交平台"

✅ "创建一个命令行社交网络原型"

2. 提供上下文

- ✗ "创建一个博客系统"
- ✓ "创建一个面向技术博主的博客系统，使用Markdown存储"

3. 迭代完善

第一步: "创建一个基本的Flask API"
第二步: "添加用户认证功能"
第三步: "添加分页支持"

4. 明确验收标准

"这个系统需要能够：
1. 处理1000个并发请求
2. 支持用户登录
3. 数据持久化到PostgreSQL"

九、相关资源

- Skill 定义: [.trae/skills/vibe-project-generator/SKILL.md](#)
- GraphRAG 示例: [local_graphrag_ollama.py](#)、[local_graphrag_neo4j.py](#)
- 项目指南: [PROGRAMS_GUIDE.md](#)
- 原理详解: [GRAPH_RAG_EXPLANATION.md](#)

十、快速开始

现在就可以尝试！告诉大模型你想要开发什么：

"我想开发一个..."

大模型会自动检测需求并调用 Vibe Project Generator Skill 为你生成完整的项目！

文档版本：v1.0 更新日期：2026年5月